Problem 1. Consider the following type declarations
TYPE
A1 : integer;
A2 : pointer to float;
A3 : pointer to integer;
T1 : structure { x : integer; }
T2 : structure { x : A1* next : pointer to integer; }
T3 : structure { a : integer; b : float; }
T4 : structure { b : float; a : integer; }
T5 : structure { a : pointer to T5; b : pointer to T6; c : pointer to T7; }
T6 : structure { a : pointer to T6; b : pointer to T5; c : pointer to T5; }
T7 : structure { a : pointer to T6; b : pointer to T7; c : pointer to T9; }
T8 : structure { a : pointer to T7; b : pointer to T6; c : pointer to T10; }
T9 : array [4][5] of T8; // array 4 rows 5 columns
T10 : array [4][5] of T7;
Assuming the most permissive definition of structural equivalence, which types are structurally equivalent?

With the assumption of the most permissive definition of structural equivalence, these types are structurally equivalent:

T5 and T6 because they have the same number of the same types in the same order.

T7 and T8 because they have the same number of the same types in the same order.

T9 and T10 because the array has the same dimensions and T8 and T7 are equal.

	A1	A2	A3
A1	T	F	F
A2	F	T	F
A3	F	F	T

	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10
T1	T	F	F	F	F	F	F	F	F	F
T2	F	T	F	F	F	F	F	F	F	F
T3	F	F	T	F	F	F	F	F	F	F
T4	F	F	F	T	F	F	F	F	F	F
T5	F	F	F	T	T	F	F	F	F	F
T6	F	F	F	F	T	T	F	F	F	F
T7	F	F	F	F	F	T	T	F	F	F
T8	F	F	F	F	F	F	T	T	F	F
T9	F	F	F	F	F	F	F	F	T	T
T10	F	F	F	F	F	F	F	F	T	T

Consider the following variable declarations in conjunction to the above type declarations
VAR

s : T9;
t : T9;
u : T10;
v : array [5][4] of T8;
w, z : struct {
int a;
struct T5* next;
// var declaration section
};
x, y : struct {
int a;
struct T5* next;
};
f : function of T9 returns int;
g : function of T9 returns A1;
m : int;
n : A1;
Assume that assignments between variables are allowed if the types of the variables are equivalent. For each of the following, list all type equivalence schemes under which the expression is valid. Consider name equivalence, internal name equivalence, and structural equivalence for each case. Assume that if two variables are equivalent under name equivalence, they are also equivalent under internal name equivalence.

- **s = t;**
- **t = u;**
- **u = v;**
- **v = w;**
- **w = z;**
- **z = x;**
- **m = f(s)**
- **n = f(u)**

These expressions are valid:

s = t because of T9 is equal to T9;

t = u because of T9 is equal to T10 through structural equivalence;

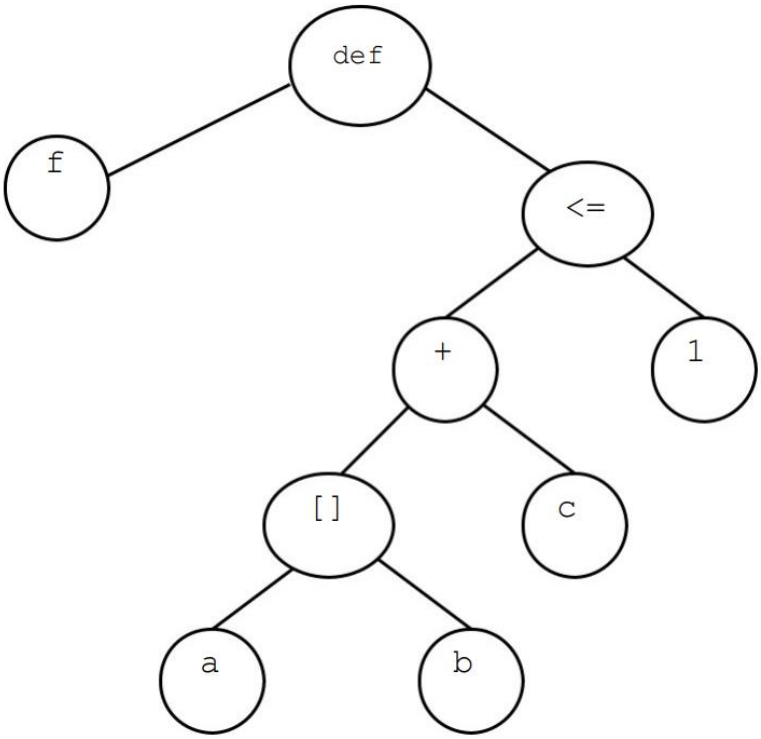
w = z because of internal name equivalence.

z = x because both have the same structure.

Problem 2. Consider the following definition

fun f(a, b, c) = a[b] + c <= 1

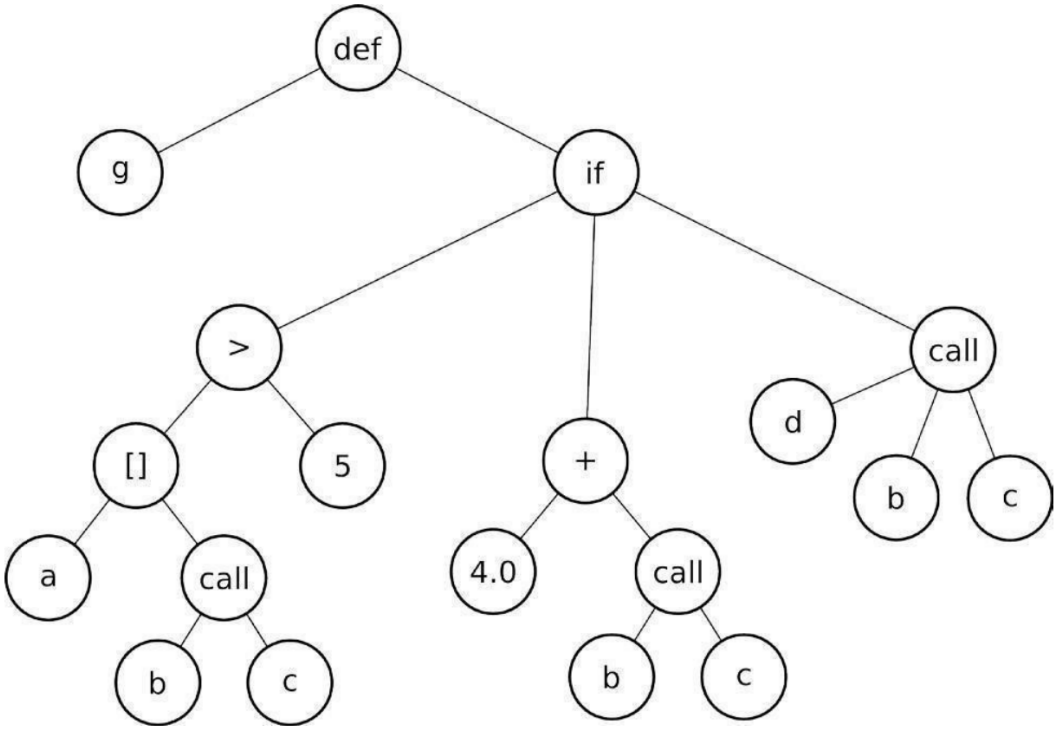
Using Hindley-Milner type inference, determine the type of f.



Using Hindley-Milner type inference, the type of f is bool. This is due to fact <= 1 is part of the expression. Since it's part of the expression, it's a Boolean type because you're checking if a[b] + c is less or equal to one. 5. The result of the expression a[b] + c is compared to 1 using the less than or equal operator, which requires a Boolean result. Therefore, the return type of the f is bool.

Problem 3. Consider the following definition:
fun g(a, b, c, d) = if a[b(c)] > 5 then
4.0 + b(c)

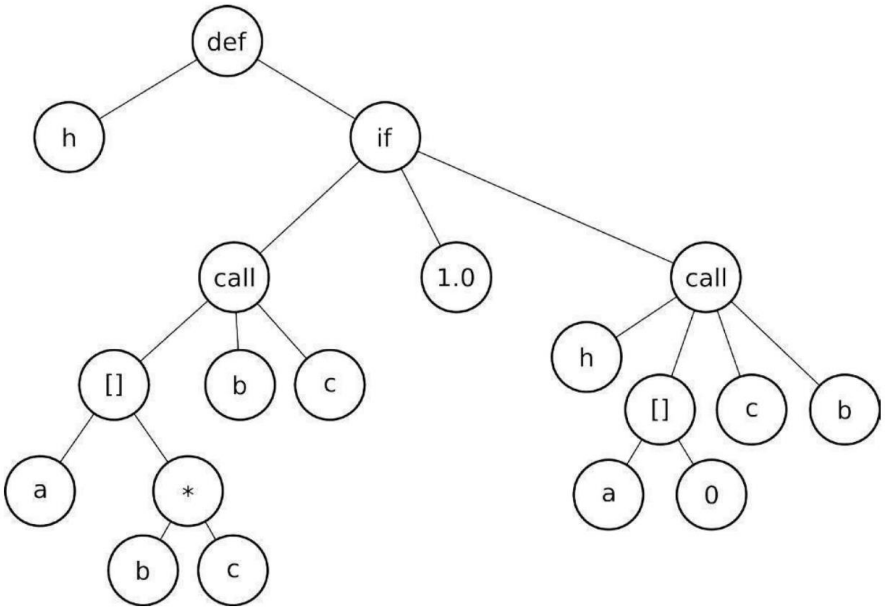
else
d(b, c)
Using Hindley-Milner type inference, determine the type of g.



Using Hindley-Milner type inference, the type of g is real. We know a is an array, d and b are function, and 4.0 is a real number. We do not know the type of c or the return type of d. We do know b's return type is an int. Therefore, the return type of g is real because an int plus a real number is a real number.

Problem 4. Consider the following definition:
fun h(a, b, c) = if a[b * c](b, c) then
1.0

else
h(a[0], c, b)
Using Hindley-Milner type inference, determine the type of h.



Using Hindley-Milner type inference, the type of h is real. This is due to the 1.0 in the function. The type of a is an array, and b and c are both int. The functions h(a, b, c) and h(a[0], c, b) led to 1.0 making the return type a real number. Therefore, the type of h is real